

Enterprise Architecture Specification: Dual-Sided Restaurant Reservation, Floor Management, and Guest Intelligence Platform

Architectural Paradigm and Structural Topology

Modern food and beverage operations depend on immediate, bidirectional synchronization between front-of-house table inventory, kitchen fulfillment pacing, and consumer booking demand¹. Traditional hospitality management systems operate as fragmented silos: reservation software functions independently of on-premise point-of-sale hardware, floor managers track table turnover using static paper sheets or rudimentary timers, and customer contact data remains trapped in proprietary third-party reservation marketplaces¹. This architectural disconnect deprives operators of first-party guest ownership, causes seating bottlenecks during peak dining hours, and limits operational throughput².

The architecture specified herein establishes an integrated, dual-sided platform designed to resolve these inefficiencies. The presentation tier decouples restaurant operational controls from consumer interactions while binding both domains to a unified relational state engine¹. The merchant administrative console operates as an on-premise operational terminal, providing floor hosts and general managers with real-time command over table occupancy, seating allocations, kitchen ticket pacing, and administrative approval queues¹. Concurrently, the consumer application functions as an inventory-aware discovery, booking, and membership funnel¹. Diners explore curated venues by administrative locality, verify real-time table availability across standardized service shifts, track reservation confirmations, and engage with personalized loyalty rewards¹.

Underlying both interfaces is an enterprise data and integration bus that unifies five operational domains:

- Physical table inventory and dynamic turn tracking¹
- Front-of-house reservation triage and algorithmic seating allocation¹
- Captive Wi-Fi network onboarding and identity resolution³
- Point-of-sale (POS) ticket commerce, itemized spend auditing, and void analysis⁶
- A centralized customer relationship management (CRM) and tiered membership engine⁴

The platform transition begins with a deterministic core—enforcing strict thirty-minute

turnaround intervals ($\Delta t_{TAT} = 30 \text{ minutes}$), fixed neighborhood locality filtering, and database-level row locking—before scaling into an event-driven production environment featuring machine-learning yield pacing, geocoded spatial indexing, and distributed cache locking¹.

Architectural Domain	Operational Mandate	Initial Core Pattern	Production Target Architecture
Merchant Administration	Shift hours, seating layouts, and live availability toggling ¹ .	Direct web-based configuration forms with reactive state updates ¹ .	Multi-tenant organization hierarchy, role-based access control, and shift audit logs ¹ .
Floor State Engine	Real-time monitoring of table occupancy, orders, and service timers ¹ .	Tabular grid dashboard with client-side interval synchronization ¹ .	Bidirectional WebSocket vector canvas with distributed publish-subscribe synchronization ¹ .
Consumer Booking	Geographic discovery, inventory querying, and booking submissions ¹ .	Static administrative locality dropdown with restaurant profile cards ¹ .	Reverse-geocoded spatial search with dynamic availability indexing ¹ .
Concurrency & Integrity	Elimination of race conditions and double-allocation anomalies ¹ .	Relational row-level locking (SELECT FOR UPDATE) over discrete windows ¹ .	Distributed Redis key leases with millisecond-level time-to-live expirations ¹ .
Guest Intelligence & CRM	Profile enrichment, visit habits, and behavioral tagging ⁹ .	Normalized relational profile records capturing basic contact data ¹ .	Unified Customer Data Platform (CDP) computing dynamic auto-tags and lifetime value metrics ⁹ .
Physical Network Ingestion	Frictionless in-venue diner acquisition and presence verification ³ .	Informational manual entry forms.	Hardware-integrated captive Wi-Fi portal with automatic MAC resolution and

			RADIUS handshakes ³ .
Commerce & POS Integration	Live ticket pacing, spend capture, and financial audits ¹ .	Mock order tracking with client-side elapsed timers and manual ticket states ¹ .	Bidirectional webhooks connecting POS terminals for itemized ordering, voids, comps, and gift cards ⁶ .
Loyalty & Membership	Capital float preloading, spend multipliers, and redemptions ⁴ .	Static pricing benchmarks (cost_for_one, cost_for_two) ¹ .	Multi-tiered membership engine with double-entry cryptographic ledger balances and in-app checkout ⁴ .

Market Benchmark Analysis: Competitive Feature Synthesis

To deliver immediate competitive advantages over legacy reservation utilities and specialized restaurant CRMs, the platform synthesizes leading architectural patterns from Resy, SevenRooms, Toast, and Blackbird⁴.

Unified Guest Profile (CDP)

Demographic Details: John Doe | +1 (555) 019-2834 | john.doe@example.com

Behavioral Auto-Tags: [VIP: Tier 3] [Top Spender] [Pacing: Slow] [Gluten Allergy]

Seating Preferences: Booth Requested | Corner Table | Quiet Zone | Avoid Entrance

Historical Metrics: Visits: 14 | Avg Party: 3.2 | Total Spend: \$4,250 | Voids: 1 (\$42.00)

POS Item Affinities: Dry-Aged Ribeye (x8) | Cabernet Sauvignon (x12) | Fresh Oysters (x6)

Hardware Identifiers: Wi-Fi MAC: 3A:8F:C2:11:09:DE | Last Connection: 2024-03-28 19:42

Financial Balances: Loyalty: 4,500 Fly Points (\$45.00) | Stored Gift Card: \$150.00

Guest Profile Enrichment, Preference Modeling, and Auto-Tagging

Legacy booking engines treat customer records as ephemeral transaction logs, discarding behavioral nuances between dining sessions². Modern hospitality platforms such as SevenRooms and Resy demonstrate that operational differentiation depends on rich, persistent guest intelligence⁹. Guest profiles must capture more than names and contact numbers; they must aggregate visit frequency, average party size, lifetime spend, dining preferences, and specific culinary restrictions².

Automated tagging systems remove data entry burdens from floor staff by analyzing transactional data streams in real time². When an incoming order contains high-margin vintage wines or premium dry-aged steaks, the platform associates tags like Wine Connoisseur or High Spender with the diner's profile⁹.

Similarly, dining durations update behavioral tags: tables consistently occupying seats beyond standard turnaround windows are flagged as Slow Pacing, allowing reservation algorithms to adjust future buffer thresholds⁹. Critical dining constraints, such as severe medical allergies, are explicitly separated from soft preferences (such as window seating or booth requests), ensuring that health-critical alerts are prominently displayed on host terminals and printed directly on kitchen chits¹³.

Frictionless Captive Wi-Fi Ingestion

Acquiring complete guest intelligence is often hindered by the "single-booker blindspot"³. In typical dining reservations, a single host reserves a table for four to eight people, leaving the restaurant with no digital relationship or contact information for the remaining guests³.

Implementing a captive Wi-Fi architecture solves this challenge by serving as an on-premise identity capture mechanism³.

When arriving guests connect to the dining room Wi-Fi network, the captive portal intercepts the initial HTTP request and displays a branded authentication interface³. Guests gain high-speed network access by entering their first name, last name, mobile number, and marketing consent parameters, adhering strictly to global privacy frameworks such as GDPR and CCPA³.

The system records the device Media Access Control (MAC) address and creates or updates a verified profile in the centralized CRM³. The platform then matches this physical presence against the reservation run-sheet, alerting the host stand that an expected party has entered the building even before they approach the reception desk³.

Omnichannel Membership Programs and Preloaded Capital Floats

Customer retention within the restaurant industry is evolving beyond basic, retroactive point systems⁴. Web3 dining networks such as Blackbird demonstrate the value of upfront

membership status, digital wallet integration, and spend-based incentives⁴. The platform implements an advanced membership architecture featuring multi-tiered progression:

- Base Member
 - Club Regular
 - Friends & Family
- [cite: 4, 12]

Tier status can be unlocked through two pathways: cumulative annual dining expenditure or single-instance wallet preloading⁴. For example, depositing \$250 directly into an in-app wallet immediately elevates a guest to Club Regular, bypassing spend accumulation requirements⁴. This approach provides restaurant operators with immediate, non-dilutive working capital while incentivizing repeat visits⁴.

Dynamic point multipliers scale alongside membership tiers, granting elite patrons between 4x and 7x points on eligible dining purchases¹². Loyalty points function as a liquid currency pegged to standard monetary units (e.g., 100 points = \$1.00 USD), allowing diners to redeem rewards directly against point-of-sale folios within the mobile app⁴. Furthermore, elite tiers unlock exclusive operational privileges, including priority access to prime reservation slots, off-peak table holds, and dedicated concierge messaging channels⁴.

Itemized Point-of-Sale Integration, Voids, Comps, and Gift Cards

A comprehensive restaurant management platform requires deep synchronization with commercial point-of-sale hardware, including Toast, NCR Aloha, and Micros Symphony⁶. Direct POS integration eliminates discrepancies between floor operations and financial reconciliation². When a party is seated, the host terminal opens a POS check tied to that specific physical table and reservation record¹. As waitstaff enter food and beverage orders, itemized lines stream to the core platform, updating kitchen preparation statuses and tracking service pacing¹. Capturing transaction exceptions—specifically item voids, promotional managerial comps, and dish returns—is vital for behavioral tracking and fraud prevention⁷. If a kitchen mistake or guest rejection occurs, the POS emits a webhook detailing the void reason code, authorized manager ID, and total monetary deduction⁸. The platform adjusts the running check balance, records the event in an audit log, updates the guest profile to track void frequencies, and recalculates loyalty point accruals to prevent issuing rewards on non-settled spend⁷. The commerce engine also maintains an integrated gift card ledger⁶. Guests can purchase, reload, and redeem digital or physical gift cards across web portals, mobile apps, and table-side terminals⁶. Stored balances update automatically across all channels, providing operators with clear liability reporting while giving diners flexible payment options⁷.

Functional Pillar	Resy Operational	SevenRooms Operational	Toast Ecosystem Benchmark	Blackbird Hospitality Benchmark	Integrated Platform Production
-------------------	------------------	------------------------	---------------------------	---------------------------------	--------------------------------

	Benchmark	Benchmark			Standard
Guest Profile & CRM	Basic guest profiles, visit frequency tracking, and cancellation metrics ¹³ .	Over 100 auto-tags, item preferences, and cross-venue profile sync ⁹ .	Transactional spend history and linked credit card indexing ¹³ .	Dynamic tier scores, visit counters, and dining persona tags ¹⁵ .	Automated auto-tagging, preference profiles, dietary tracking, and cross-venue sync ⁹ .
Physical Identification	Manual host check-in and mobile notification links ¹³ .	Captive Wi-Fi connectors syncing verified diner contact data ³ .	Credit card token matching at POS card readers ¹³ .	Near-Field Communication (NFC) check-in pucks at host stands ¹² .	Multi-source ingestion: Captive Wi-Fi MAC mapping, host-stand NFC taps, and card token indexing ³ .
Loyalty & Membership	Co-branded American Express card priority access ¹² .	Custom perks, surprise-and-delight upgrades, and corporate tier tags ⁹ .	Traditional card-linked loyalty points and flat reward discounts ¹³ .	Tiered crypto/fiat membership floats, preloads, and 3x-7x multipliers ⁴ .	Tiered membership with wallet preloads, dynamic point multipliers, and in-app folio redemptions ⁴ .
POS Commerce Integration	Read-only check closure hooks and manual	Bidirectional spend capture, live spend dashboards	Native, unified kitchen display, order firing,	External payment rails connecting to POS	Full bidirectional POS sync: itemized firing, void

	table clearing ¹ .	, and item history ⁶ .	and folio balancing ⁶ .	checks via partner APIs ²¹ .	tracking, comp logging, and gift card validation ⁶ .
Turnover Engine	Variable booking windows based on static table shift rules ¹ .	Dynamic pacing, table optimization algorithms, and live waitlists ¹⁰ .	Native server pacing and kitchen display timers ⁸ .	Post-dining payment triggers and automated table release ⁴ .	Real-time elapsed timers, predictive turn pacing, and automated table status updates ¹ .

End-to-End Operational Workflows and System Interactions

Consumer Discovery, Real-Time Inventory Querying, and Hold Allocation

The consumer booking journey balances low-friction discovery with rigid transactional concurrency controls to eliminate double-booking anomalies¹.

The process initiates when the consumer opens the web portal or mobile application and selects an administrative locality via the global header dropdown¹. This deterministic selector queries the directory service, filtering out inactive establishments or venues outside the target municipal sector¹. The client displays structured restaurant profile cards featuring key discovery metrics: culinary category, aggregate diner reviews, operational status, and pricing benchmarks (cost_for_one and cost_for_two)¹.

Upon selecting a dining establishment, the consumer enters party size P , target calendar date D , and preferred arrival time T_{target} into a fifteen-minute slot selector¹. The client dispatches an availability query to the backend reservation engine¹. The system discretizes the venue operating shift (T_{open} to T_{close}) and evaluates table inventory assets¹.

To determine whether an inventory asset can support the request, the engine checks for interval overlap against all confirmed, seated, or requested reservations tied to that physical table¹. A scheduling collision exists if and only if:

$$\max(S_e, S_n) < \min(E_e, E_n)$$

where $[S_e, E_e)$ denotes an existing active booking interval and $[S_n, E_n)$ denotes the requested dining duration $[T_{start}, T_{start} + \Delta t_{TAT})$ ¹.

Tables returning no interval overlap that satisfy structural seating constraints ($C_{min} \leq P \leq C_{max}$) are returned to the client as available booking slots¹.

When the diner selects a validated slot and inputs their contact details (full name, phone number, email address, dietary restrictions, and seating preferences), the application initiates an atomic reservation hold¹. The database opens a transaction, acquires an exclusive row-level lock on candidate table records, and inserts a reservation in the REQUESTED state¹.

This holds the table asset for a predefined duration, preventing concurrent threads from claiming the same inventory¹. The booking payload routes to the merchant administrative triage queue, while the consumer interface transitions to a real-time status screen awaiting host confirmation¹.

Merchant Floor Management, Host Triage Queue, Seating, and Live Table Turnover

The merchant floor console provides hosts and general managers with centralized oversight of dining room operations, balancing digital reservation requests with in-person walk-ins¹.

	Merchant Floor Console: Operational Service Timeline
	[18:00] Digital Booking Arrives -> Added to Host Triage Queue (Status: REQUESTED)
	[18:02] Host Reviews Guest Tags & Notes -> Clicks 'Approve' (Status: CONFIRMED, Table RESERVED)
	[18:30] Party Arrives at Host Stand -> Host Clicks 'Seat Party' (Status: SEATED, Table OCCUPIED)
	[18:32] POS Ticket Opened -> Kitchen Elapsed Timer Starts Counting ($\Delta t_{elapsed} = t_{current} - t_{seated}$)
	[19:00] Facing Alert Triggered -> Timer Reaches 30 Mins -> UI Tile Flashes Amber/Red
	[19:05] Folio Settled on POS -> Host Marks Vacated -> Table Shifts to BUSSING -> AVAILABLE

Incoming online bookings populate the host triage queue in real time¹. The console displays the guest's profile, party size, target seating time, dietary notes, and system-generated auto-tags (such as VIP, Regular, or Late Cancellor)¹.

The floor supervisor can confirm or reject the request:

- Confirming updates the reservation to CONFIRMED, locks the table asset in RESERVED status, and dispatches a confirmation notification to the guest¹.
- Rejecting the request releases the inventory lock, marks the reservation CANCELLED, and prompts the diner to select an alternate dining window¹.

When a confirmed party arrives within the fifteen-minute operational grace period, the host seats the guests and updates the table status to OCCUPIED on the console¹. This seating event updates the reservation record to SEATED, creates an active order ticket container linked to the table, and initializes a client-side elapsed service timer¹:

$$\Delta t_{elapsed} = t_{current} - t_{placed}$$

The table tile renders an active, ticking counter that synchronizes against the central server clock, maintaining temporal drift below one second ($|\Delta t_{drift}| < 1.0 \text{ s}$)¹.

To support timely table turnover, the visual indicator incorporates an operational alert threshold¹. If the service duration reaches or exceeds the allocated turnaround window ($\Delta t_{elapsed} \geq 30 \text{ minutes}$

in the baseline operational profile), the tile transitions from neutral typography to a flashing amber or red alert¹.

This visual cue notifies floor hosts that the party has exceeded the standard dining duration, prompting staff to present the bill or prepare the table for incoming reservations¹. Once the check is settled and the table sanitized, the host marks the asset cleared, transitioning the unit to AVAILABLE inventory¹.

Operational Stage	Host Console Interaction	Background System Execution	UI Status Indicator
Request Ingestion	Incoming reservation appears in host queue ¹ .	Inserts pending reservation record; verifies slot availability ¹ .	Orange badge in triage queue ¹ .
Host Approval	Host clicks Approve control ¹ .	Sets status to CONFIRMED;	Table tile shifts to Controlled Amber ¹ .

		transitions table to RESERVED ¹ .	
Party Check-In	Host clicks Seat Party on arrival ¹ .	Sets status to SEATED; transitions table to OCCUPIED; records t_{placed} ¹ .	Table tile shifts to Operational Blue; timer runs ¹ .
Service Pacing	Floor monitors dining duration ¹ .	Increments $\Delta t_{elapsed}$; syncs clock drift across host consoles ¹ .	Digital timer displays standard typography ¹ .
Turnover Alert	Dwell time reaches threshold ($\Delta t_{elapsed} \geq$) ¹ .	Flags table for host attention; evaluates floor capacity for upcoming bookings ¹ .	Timer shifts to flashing Amber/Red alert ¹ .
Folio Settlement	Host confirms payment and table sanitization ¹ .	Closes active ticket; archives session; reverts table to AVAILABLE ¹ .	Table tile returns to High-Visibility Green ¹ .

Captive Wi-Fi Onboarding and Automated Guest CRM Synchronization

The captive Wi-Fi integration functions as an automated identity-capture engine, capturing verified guest information to enrich CRM records³.

The onboarding sequence begins when a diner connects to the restaurant's public wireless network³. The network access point intercepts browser traffic and redirects the device to the captive portal landing page, appending the client hardware MAC address and access point ID to the URL query string³.

The portal presents a lightweight onboarding form requiring the diner's first name, last name, mobile phone number, email address, and explicit marketing opt-in consent³.





When the user submits the form, the platform executes an identity-resolution routine³:

1. The CRM checks the database for existing records matching the submitted phone number or email address³.
2. If a match is found, the system links the device MAC address to the existing profile, appends a new visit timestamp, and increments the guest's visit counter³.
3. If no match exists, the system provisions a new guest record, stores the device identifiers, and assigns baseline membership status³.
4. The authentication service sends an Access-Accept authorization command to the network controller via RADIUS, granting the guest full internet access³.

Simultaneously, the CRM evaluates active reservations for the current shift³. If the authenticating diner matches a confirmed reservation or is recognized as an elite membership tier, the merchant floor console displays an instant arrival chit: Guest Connected: [Name], Tier: Club Regular, Table: T-4, Prefers Quiet Corner³.

This automation alerts floor staff to the patron's arrival before they check in at the host stand, enabling personalized greetings and tailored hospitality³.

Bidirectional Point-of-Sale Integration, Item Voids, Comps, and Gift Cards

The commerce engine coordinates transactions between floor operations, kitchen display systems, and point-of-sale hardware¹.

Seating a party on the host console automatically opens an active check container on the POS terminal, linking the table identifier and reservation reference¹. As waitstaff enter food and beverage orders, the POS emits a ticket.updated event containing itemized dish names, product categories, quantities, unit prices, and kitchen modification instructions⁶.

The platform stores these line items in the relational database, links them to the active dining session, and recalculates the running balance¹. Kitchen preparation times update the operational console, ensuring front-of-house staff have continuous visibility into order fulfillment pacing¹.

When a transaction exception occurs—such as a dish return, kitchen error, or managerial discount—the POS transmits an order.item_voided or order.comp_applied webhook⁸. The incoming payload includes the item ID, manager authorization badge, reduction value, and mandatory void reason string (e.g., GUEST_REJECTED, PREPARATION_DEFECT, PROMOTIONAL_COMP)⁸.

The system processes this exception through several automated steps:

- The monetary value is deducted from the active check balance⁷.
- The void event is recorded in a dedicated audit log table (pos_void_logs)⁸.
- The guest's CRM profile updates its void tracking metrics (total_voids_count and total_voids_value), flagging patterns of frequent complaints for managerial review⁸.
- The net settled spend basis for loyalty point accrual is reduced, ensuring rewards are calculated solely on realized revenue⁷.

Folio settlement also supports stored-value gift cards⁶. When a customer presents a gift card, the POS or consumer mobile app issues a balance check and authorization request: POST /api/v1/gift-cards/authorize⁷.

The platform verifies the card hash, checks that available funds cover the requested charge, places a temporary lock on the balance, and returns an approval token⁷. Upon check closure, the authorization settles, writing a debit entry to the immutable gift card ledger and clearing the check balance on the POS⁷.

Dynamic Membership Tier Progression, Preloaded Wallets, and In-App Redemption

The customer application functions as an active digital membership card and payment wallet, driving repeat visits through tiered incentives⁴.

Diners advance through membership tiers via two distinct mechanisms:

- Accumulating qualifying dining spend over a calendar year⁴
- Depositing capital directly into their in-app wallet⁴

For example, reaching the Club Regular tier requires either spending \$5,000 across multiple dining visits or loading \$2,500 into the digital wallet in a single transaction⁴. Deposited funds remain in the diner's wallet as liquid capital for future visits, while providing the restaurant with non-dilutive working capital upfront⁴.

Dynamic point multipliers scale alongside membership tiers¹². When an order is settled, the loyalty engine ingests the net food and beverage total (excluding tips, taxes, voids, and comps) and applies the member's current tier multiplier⁴:

- Base Member: 3x points per dollar spent¹²
- Club Member: 4x points per dollar spent⁴
- Club Regular: 5x points per dollar spent⁴
- Friends & Family: 7x points per dollar spent¹²

Loyalty points function as spending power pegged at a standard conversion rate of 100 points per \$1.00 USD⁴. During in-app payment or table-side mobile checkout, diners can toggle Apply Points Balance to offset check totals⁴. The system applies the points credit, debits the member's points ledger, and routes any remaining balance to the linked payment method⁴.

Membership Tier Level	Spend Qualification Threshold	Alternative Wallet Preload	Base Dine-In Multiplier	Specialized Privilege Entitlements
Base Member	Default upon onboarding ⁴ .	\$0.00 ⁴ .	3x Points per \$1.00 ¹² .	Standard digital booking access and automated Wi-Fi check-in ³ .
Club Member	\$500.00 settled spend ⁴ .	\$250.00 wallet deposit ⁴ .	4x Points per \$1.00 ⁴ .	10x earn rate at partner cafes and priority waitlist placement ⁴ .
Club Regular	\$5,000.00 settled spend ⁴ .	\$2,500.00 wallet deposit ⁴ .	5x Points per \$1.00 ⁴ .	Prime-time booking access, concierge support, and Members' Night invites ⁴ .

Friends & Family	\$10,000.00 settled spend ⁴ .	\$5,000.00 wallet deposit ⁴ .	7x Points per \$1.00 ¹² .	24/7 dedicated concierge, off-peak table holds, and kitchen tasting upgrades ⁹ .
-----------------------------	--	--	--------------------------------------	---

Domain Data Architecture and Normalized Relational Models

The platform uses a normalized relational data model designed to preserve ACID transactional guarantees under heavy concurrency, with flexible JSONB extensions for dynamic guest preferences and device footprints¹.

Venue, Shift, and Table Inventory Domain

The physical floor layout is modeled as a hierarchy of addressable assets constrained by capacity boundaries, preventing abstract aggregate headcounts from causing seating bottlenecks¹.

Table Name	Column Name	Storage Type	Database Constraints	Operational Role & Business Rules
venues	id	UUID	Primary Key, gen_random_uuid()	Unique identifier for physical dining venues ¹ .
venues	name	VARCHAR(255)	NOT NULL	Trade name of the restaurant ¹ .
venues	locality_id	UUID	Foreign Key (localities.id), NOT NULL	Locality used for static dropdown filtering ¹ .
venues	cost_for_one	DECIMAL(10,2)	NOT NULL, Check (cost_for_one > 0)	Benchmark individual spend metric ¹ .

venues	cost_for_two	DECIMAL(10,2)	NOT NULL, Check (cost_for_two > 0)	Benchmark couple dining metric ¹ .
venues	turnaround_time_mins	INT	NOT NULL, Default 30	Base turnaround duration constant (Δt_{TAT}) ¹ .
venues	is_active	BOOLEAN	NOT NULL, Default TRUE	Global operational toggle between active booking and blackout modes ¹ .
operating_shifts	id	UUID	Primary Key	Identifier for daily operating shifts ¹ .
operating_shifts	venue_id	UUID	Foreign Key (venues.id), NOT NULL	Associated venue location ¹ .
operating_shifts	day_of_week	INT	NOT NULL, Check (day_of_week BETWEEN 0 AND 6)	0 = Sunday, 6 = Saturday schedule mapping ¹ .
operating_shifts	open_time	TIME	NOT NULL	Daily shift opening timestamp T_{open} ¹ .
operating_shifts	close_time	TIME	NOT NULL	Daily shift closing

				timestamp T_{close}^1 .
tables	id	UUID	Primary Key	Identifier for physical table assets ¹ .
tables	venue_id	UUID	Foreign Key (venues.id), NOT NULL	Venue containing the physical table ¹ .
tables	table_number	VARCHAR(32)	NOT NULL	Visual table label (e.g., "T-12") ¹ .
tables	dining_zone	VARCHAR(64)	NOT NULL, Default 'MAIN'	Floor zone: 'MAIN', 'PATIO', 'BAR', 'PRIVATE' ¹ .
tables	min_capacity	INT	NOT NULL, Check (min_capacity >= 1)	Minimum party size constraint C_{min}^1 .
tables	max_capacity	INT	NOT NULL, Check (max_capacity >= min_capacity)	Maximum seating capacity constraint C_{max}^1 .
tables	current_status	ENUM	Available, Reserved, Occupied, Bussing, Blocked	Real-time operational table asset state ¹ .

Guest Intelligence, CRM, and Tagging Domain

The guest intelligence schema stores behavioral profiles, auto-generated tags, dietary flags, and aggregate POS spend histories².

Table Name	Column Name	Storage Type	Database Constraints	Operational Role & Business Rules
guests	id	UUID	Primary Key	Canonical diner identity record ¹ .
guests	first_name	VARCHAR(128)	NOT NULL	Guest legal first name ¹ .
guests	last_name	VARCHAR(128)	NOT NULL	Guest legal last name ¹ .
guests	phone_number	VARCHAR(32)	Unique, NOT NULL	Primary key for SMS verification and CRM matching ¹ .
guests	email	VARCHAR(255)	Unique, NOT NULL	Primary key for confirmations and email marketing ¹ .
guest_profiles	guest_id	UUID	Primary Key, Foreign Key (guests.id)	One-to-one behavioral profile extension ⁹ .
guest_profiles	lifetime_spend	DECIMAL(12,2)	NOT NULL, Default 0.00	Net cumulative POS dining expenditure ⁹ .
guest_profiles	total_visits	INT	NOT NULL, Default 0	Counter incremented on completed visits ⁹ .
guest_profiles	avg_party_size	DECIMAL(4,2)	NOT NULL, Default 1.00	Rolling historical

				group size average.
guest_profiles	total_voids_count	INT	NOT NULL, Default 0	Number of returned or voided items ⁸ .
guest_profiles	total_voids_value	DECIMAL(10,2)	NOT NULL, Default 0.00	Cumulative value of voided items ⁸ .
guest_profiles	seating_preference	VARCHAR(64)	Default 'ANY'	Preferred seating: 'BOOTH', 'WINDOW', 'QUIET' ¹³ .
guest_profiles	dietary_notes	TEXT	Nullable	Medical allergy warnings and dietary requests ¹³ .
guest_tags	id	UUID	Primary Key	Record of behavioral tags assigned to guest ⁹ .
guest_tags	guest_id	UUID	Foreign Key (guests.id), NOT NULL	Target guest receiving classification ⁹ .
guest_tags	tag_name	VARCHAR(64)	NOT NULL	Tag label: 'VIP', 'TOP_SPENDER', 'REGULAR' ¹⁰ .
guest_tags	is_auto_generated	BOOLEAN	NOT NULL, Default TRUE	Distinguishes system auto-tags from manual staff tags ² .

Reservation and Seating Domain

The reservation ledger records table inventory claims for specific operational intervals, enforcing transactional integrity during scheduling¹.

Table Name	Column Name	Storage Type	Database Constraints	Operational Role & Business Rules
reservations	id	UUID	Primary Key	Unique transaction record for dining hold ¹ .
reservations	venue_id	UUID	Foreign Key (venues.id), NOT NULL	Target dining venue establishment ¹ .
reservations	table_id	UUID	Foreign Key (tables.id), Nullable	Assigned table; nullable during waitlist triage ¹ .
reservations	guest_id	UUID	Foreign Key (guests.id), NOT NULL	Reserving patron identity reference ¹ .
reservations	party_size	INT	NOT NULL, Check (party_size >= 1)	Party count validated against table capacity boundaries ¹ .
reservations	booking_date	DATE	NOT NULL	Calendar date of scheduled seating ¹ .
reservations	start_time	TIME	NOT NULL	Scheduled arrival timestamp T_{start} ¹ .

reservations	end_time	TIME	NOT NULL	Calculated release boundary $T_{release}^1$.
reservations	status	ENUM	Requested, Confirmed, Seated, Completed, Cancelled, NoShow	Current transactional state of booking ¹ .
reservations	created_at	TIMESTAMP	NOT NULL, Default clock_timestamp()	Insertion time used for host queue ordering ¹ .

Point-of-Sale, Orders, and Stored Value Domain

The POS integration schema captures real-time kitchen ticket progressions, itemized order data, manager-authorized voids, and gift card balances¹.

Table Name	Column Name	Storage Type	Database Constraints	Operational Role & Business Rules
pos_orders	id	UUID	Primary Key	Transactional order ticket session ¹ .
pos_orders	table_id	UUID	Foreign Key (tables.id), NOT NULL	Physical table hosting the dining check ¹ .
pos_orders	reservation_id	UUID	Foreign Key (reservations.id), Nullable	Linked booking (nullable for walk-ins) ¹ .
pos_orders	pos_external_id	VARCHAR(128)	Unique,	Check reference from

	d		Nullable	external POS system ⁶ .
pos_orders	placed_at	TIMESTAMP	NOT NULL	Initial ticket firing timestamp t_{placed}^1 .
pos_orders	settled_at	TIMESTAMP	Nullable	Final check settlement timestamp ¹ .
pos_orders	net_amount	DECIMAL(10,2)	NOT NULL, Default 0.00	Net bill total used for loyalty point accrual ⁷ .
pos_orders	order_status	ENUM	Placed, Received, Preparing, Served, Billed, Voided	Operational fulfillment state of ticket ¹ .
pos_order_items	id	UUID	Primary Key	Line item identifier ¹ .
pos_order_items	order_id	UUID	Foreign Key (pos_orders.id), NOT NULL	Associated ticket header ¹ .
pos_order_items	item_name	VARCHAR(255)	NOT NULL	Description of menu item ¹ .
pos_order_items	category	VARCHAR(64)	NOT NULL	Category: 'ENTREE', 'BEVERAGE', 'WINE' ⁸ .
pos_order_items	quantity	INT	NOT NULL, Check (quantity > 0)	Number of units ordered ¹ .

pos_order_items	unit_price	DECIMAL(10,2)	NOT NULL	Unit menu price charged ¹ .
pos_void_logs	id	UUID	Primary Key	Audit trail for voids, comps, and returns ⁸ .
pos_void_logs	order_item_id	UUID	Foreign Key (pos_order_items.id), NOT NULL	Specific line item modified ⁸ .
pos_void_logs	void_reason	VARCHAR(128)	NOT NULL	Reason code: 'SPILL', 'KITCHEN_ERROR' ⁸ .
pos_void_logs	authorized_by	VARCHAR(128)	NOT NULL	Badge or ID of manager authorizing change ⁸ .
pos_void_logs	amount_voided	DECIMAL(10,2)	NOT NULL	Monetary value removed from check ⁸ .
gift_cards	id	UUID	Primary Key	Stored-value voucher record ⁶ .
gift_cards	card_number_hash	VARCHAR(255)	Unique, NOT NULL	Salted SHA-256 hash of card number ⁷ .
gift_cards	current_balance	DECIMAL(10,2)	NOT NULL, Check (current_balance >= 0)	Real-time redeemable balance ⁷ .
gift_cards	purchased_by	UUID	Foreign Key	Purchasing

			(guests.id), Nullable	patron identity reference ⁷ .
--	--	--	--------------------------	---

Captive Network and Loyalty Program Domain

The network authentication and loyalty models track in-venue Wi-Fi sessions, membership tiers, preloaded funds, and immutable point ledger entries³.

Table Name	Column Name	Storage Type	Database Constraints	Operational Role & Business Rules
wifi_sessions	id	UUID	Primary Key	Record of in-venue Wi-Fi connection ³ .
wifi_sessions	guest_id	UUID	Foreign Key (guests.id), NOT NULL	Resolved guest profile ³ .
wifi_sessions	venue_id	UUID	Foreign Key (venues.id), NOT NULL	Venue hosting network connection ³ .
wifi_sessions	device_mac	MACADDR	NOT NULL	Hardware address captured at redirect ³ .
wifi_sessions	connected_at	TIMESTAMP	NOT NULL, Default clock_timestamp()	Timestamp of initial portal connection ³ .
wifi_sessions	opt_in_marketing	BOOLEAN	NOT NULL, Default FALSE	Explicit privacy consent flag ³ .
loyalty_accounts	id	UUID	Primary Key	Customer membership account ³ .

loyalty_accounts	guest_id	UUID	Unique, Foreign Key (guests.id)	Associated guest identity reference ³ .
loyalty_accounts	tier_level	ENUM	Base, Member, Regular, FriendsAndFamily	Membership tier determining multipliers ⁴ .
loyalty_accounts	points_balance	BIGINT	NOT NULL, Default 0	Available balance of liquid loyalty points ⁴ .
loyalty_accounts	wallet_cash_flow_at	DECIMAL(10,2)	NOT NULL, Default 0.00	Preloaded currency balance for dining ⁴ .
loyalty_ledger	id	UUID	Primary Key	Immutable double-entry balance log ⁴ .
loyalty_ledger	account_id	UUID	Foreign Key (loyalty_accounts.id), NOT NULL	Target loyalty account ⁴ .
loyalty_ledger	event_type	ENUM	Accrual, Redemption, Preload, Reversal, Expired	Category of points balance change ⁴ .
loyalty_ledger	points_delta	BIGINT	NOT NULL	Points added (positive) or deducted (negative) ⁴ .
loyalty_ledger	reference_id	UUID	Nullable	Linked check ID

				(pos_orders.id) or gift card ⁷ .
--	--	--	--	--

Mathematical Formulations, Pacing Engines, and Concurrency Control

Operational Shift Discretization and Pacing Capacity

Operational shifts are divided into discrete fifteen-minute intervals between the opening time

T_{open} and closing time T_{close} ¹. Table capacity and booking pacing are governed by the

standardized turnaround duration constant $\Delta t_{TAT} = 30 \text{ minutes}$ ¹. The maximum

number of non-overlapping booking intervals N_{slots} supported by an individual table during a normal shift is computed as¹:

$$N_{slots} = \left\lfloor \frac{T_{close} - T_{open}}{\Delta t_{TAT}} \right\rfloor$$

For any proposed reservation starting at timestamp T_{start} , the terminal release threshold

$T_{release}$ is calculated as¹:

$$T_{release} = T_{start} + \Delta t_{TAT}$$

The scheduling engine automatically rejects booking requests where the requested arrival time

precedes the shift opening ($T_{start} < T_{open}$) or where the release boundary exceeds the

shift closing threshold ($T_{release} > T_{close}$)¹.

Interval Overlap and Collision Detection

To verify slot availability, candidate tables are checked for conflicting bookings within the

interval $[T_{start}, T_{release})$ ¹. A scheduling collision between an existing active reservation

$R_e = [S_e, E_e)$ and a new reservation request $R_n = [S_n, E_n)$ exists if and only if¹:

$$\max(S_e, S_n) < \min(E_e, E_n)$$

If this condition evaluates to true for any active reservation tied to the table (with status Requested, Confirmed, or Seated), the table is marked conflicted and excluded from availability results for that window¹.

Best-Fit Table Allocation Heuristic

When a customer requests a reservation without selecting a specific table, the booking engine applies a best-fit allocation algorithm to assign the most appropriate table while preserving capacity for larger parties¹.

The algorithm identifies all tables in the venue that have no overlapping bookings during $[T_{start}, T_{start} + \Delta t_{TAT})$ and whose status is Available¹. It filters this set to tables where the guest count P satisfies seating constraints¹:

$$C_{min} \leq P \leq C_{max}$$

Candidate tables are then sorted in ascending order of seat differential ΔC ¹:

$$\Delta C = C_{max} - P$$

The system assigns the table with the lowest non-negative ΔC ¹. Under this rule, a party of two ($P = 2$) will match a two-seater table ($C_{max} = 2, \Delta C = 0$) rather than a four-seater table ($C_{max} = 4, \Delta C = 2$), reserving larger tables for groups of three or four diners¹.

Concurrency Race Mitigation Mechanics

High-concurrency race conditions occur when multiple patrons attempt to book the final available table for the same time slot simultaneously¹. Without concurrency controls, simultaneous requests can result in double bookings¹.

To prevent conflicting allocations without introducing complex distributed lock managers during initial phases, the system enforces concurrency control directly at the relational database level¹. When a booking request passes party-size validation, the backend opens a database transaction that acquires an exclusive row-level lock on candidate table records for the requested venue and window (SELECT id FROM tables WHERE id = candidate_id FOR UPDATE)¹.

The system queries the reservation ledger to verify whether an overlapping booking exists¹. If clear, the transaction inserts the new reservation with status Requested and commits, automatically releasing the row lock¹.

If a concurrent thread attempts to evaluate the same table for an overlapping time slot, its transaction blocks until the first completes¹. Upon acquiring the lock, the second transaction detects the newly inserted reservation, identifies the schedule conflict, rolls back, and returns an HTTP 409 Conflict error¹.

In production deployments, this database-level locking transitions to distributed Redis key leases¹. An atomic lock lease is acquired via:

SET lock:table:{table_id}:{timestamp} {token} NX PX 5000

This non-blocking approach handles surge traffic without overloading database connection pools¹.

Live Elapsed Pacing Timers and Drift Calibration

When a party is seated, the host console initiates an active elapsed dining duration counter¹:

$$\Delta t_{elapsed} = t_{current} - t_{placed}$$

Because multiple host terminals run the visual floor dashboard simultaneously, timers must synchronize against a central server clock to prevent visual inconsistencies¹. Timestamps are recorded in ISO-8601 UTC format, and client clocks synchronize against server timestamps upon every ticket lifecycle change, enforcing an upper bound on temporal drift¹:

$$\Delta t_{drift} = |t_{client_rendered} - t_{server_canonical}| < 1.0 \text{ second}$$

If $\Delta t_{elapsed} \geq 30 \text{ minutes}$, the timer display shifts from standard typography to a flashing amber or red alert, notifying floor staff that the table has exceeded the standard service turnaround window¹.

Finite State Machine Models and Operational Lifecycle Transitions

Every transactional domain in the platform follows deterministic finite state machine (FSM) transitions to enforce consistency across consumer interfaces, host terminals, and kitchen displays¹.

Reservation Lifecycle State Transitions





Originating State	Target State	Triggering Event & Condition	System Action & Interface Updates
Initial Submission	REQUESTED	Diner submits valid party size, date, and available slot ¹ .	Inserts pending reservation; displays ticket in merchant triage queue ¹ .
REQUESTED	CONFIRMED	Restaurant host clicks approval control ¹ .	Sets table state to RESERVED; dispatches confirmation notification to diner ¹ .
REQUESTED	CANCELLED	Host rejects booking or diner cancels before review ¹ .	Marks record cancelled; releases temporary inventory hold ¹ .
CONFIRMED	SEATED	Host marks party as arrived within grace period ¹ .	Sets table state to OCCUPIED; opens POS order ticket; starts elapsed timer ¹ .

CONFIRMED	NO_SHOW	Party fails to arrive within 15-minute grace period ¹ .	Reverts table state to AVAILABLE; increments guest no-show counter ¹ .
CONFIRMED	CANCELLED	Guest cancels confirmed booking prior to arrival ¹ .	Releases table inventory; logs cancellation audit record ¹ .
SEATED	COMPLETED	Host settles POS billing and confirms table vacated ¹ .	Sets table state to AVAILABLE; stops timer; triggers loyalty point accrual ¹ .

Physical Table Asset Inventory Lifecycle Transitions

Originating State	Target State	Triggering Event & Condition	Operational Implication & Visual Indicator
AVAILABLE	RESERVED	Reservation confirmed for upcoming shift interval ¹ .	Table locked for specific 30-minute block; renders Controlled Amber ¹ .
AVAILABLE	OCCUPIED	Host seats walk-in party directly ¹ .	Table occupied; initializes POS check; renders Operational Blue ¹ .
RESERVED	OCCUPIED	Host seats arriving party against confirmed booking ¹ .	Table occupied; links reservation; starts elapsed timer; renders Operational Blue ¹ .
RESERVED	AVAILABLE	Reservation cancelled or marked no-show ¹ .	Table returned to inventory pool; renders High-Visibility

			Green ¹ .
OCCUPIED	BUSSING	POS check settled and guests vacate table ¹ .	Table pending clearing and sanitization; renders Pulsing Yellow.
BUSSING	AVAILABLE	Staff confirms table is cleared and reset ¹ .	Table eligible for immediate seating; renders High-Visibility Green ¹ .
ANY	BLOCKED	Host manually takes table out of service ¹ .	Table removed from inventory (maintenance/staffing); renders Slate Gray ¹ .

Kitchen and POS Order Fulfillment Transitions

Originating State	Target State	Triggering Event & Condition	Commercial & Kitchen Action
Initial Creation	PLACED	Server or diner submits food and beverage order ¹ .	Sets t_{placed} timestamp; displays ticket on Kitchen Display System (KDS) ¹ .
PLACED	RECEIVED	Kitchen display acknowledges ticket preparation ¹ .	Sets $t_{received}$ timestamp; updates ticket status in active order drawer ¹ .
RECEIVED	PREPARING	Line cooks commence preparation of items.	Initiates item preparation timer on kitchen display ⁸ .

PREPARING	SERVED	Food expediter confirms all items delivered to table ¹ .	Updates ticket badge to served; maintains elapsed dining timer ¹ .
SERVED	BILLED	Host generates folio and registers payment ¹ .	Closes active ticket; unlocks reservation completion transition ¹ .
ANY	VOIDED	Manager authorizes full or partial check void ¹¹ .	Deducts voided amount; logs void reason and manager ID ⁸ .

Loyalty Ledger and Balance Transitions

Originating State	Target State	Triggering Event & Condition	Accounting Ledger Impact
Initial Creation	PENDING	Check settled with net spend > ⁷ .	Calculates points via tier multiplier; holds points pending fraud checks ⁴ .
PENDING	SETTLED	Settled check verified without dispute.	Credits member points_balance; writes double-entry credit ledger record ⁴ .
SETTLED	REVERSED	POS emits post-settlement void or return event ⁷ .	Debits points; prevents unearned loyalty extraction ⁷ .
SETTLED	REDEEMED	Guest applies points against in-app folio ²² .	Debits points; offsets cash payment requirement on bill ⁴ .

Programmatic Edge Cases, Failure Modes, and Technical Guardrails

To prepare code-generation environments (such as Google Gemini, Claude Code, or Anti-Gravity) for robust implementation, the following edge cases detail precise programmatic resolution strategies, failure modes, and architectural guardrails.

High-Concurrency Booking Collisions and Race Conditions

During high-demand reservation releases, multiple users may attempt to reserve the final available four-top table for the same time slot within milliseconds of each other¹.

Without proper concurrency controls, both transactions could read the table as unreserved, generating overlapping bookings and causing double-allocation anomalies¹.

The database layer prevents this failure mode by executing booking insertions within transactions using REPEATABLE READ isolation¹. The availability validation routine executes an explicit lock on candidate table assets:

```
SQL
SELECT id, min_capacity, max_capacity
FROM tables
WHERE id = candidate_table_id
FOR UPDATE
```

The transaction checks the reservation ledger for overlapping active bookings within the requested interval using the conflict condition $\max(S_e, S_n) < \min(E_e, E_n) - 1$.

The first transaction commits successfully, setting the reservation status to REQUESTED and holding the table¹. The second transaction, unblocking after the lock release, detects the newly committed reservation, triggers a rollback, and returns an HTTP 409 Conflict response with

alternative available slots ($T \pm 15 \text{ minutes}$, $T \pm 30 \text{ minutes}$)¹.

Front-of-House Seating Overrides and Walk-In Collisions

Floor hosts managing walk-in parties often assign available tables without checking whether upcoming reservations are scheduled for the same unit¹. For example, seating a walk-in party of two at Table 4 at 6:45 PM would conflict with a confirmed reservation assigned to Table 4 at 7:15 PM, given the thirty-minute turnaround duration¹.

The host terminal prevents this collision by evaluating proposed walk-in seatings against existing allocations:

$$t_{walkin_seat} + \Delta t_{TAT} > T_{confirmed_start}$$

If a conflict is detected, the terminal blocks immediate seating and displays a warning dialog: Collision Alert: Table 4 reserved for John Doe at 7:15 PM. Dwell time exceeds arrival threshold. The system runs a best-fit allocation scan across all vacant tables to identify alternative seating options matching the party size¹. If an alternative table is available, the console suggests a one-click reassignment; if no tables are vacant, the host is prompted to enact a manual override or place the walk-in on the waitlist²³.

Wi-Fi MAC Address Randomization and Identity Resolution

Modern mobile operating systems utilize private MAC addressing, generating rotating, randomized hardware MAC addresses for each network connection³.

If a system uses MAC addresses as primary keys, returning regulars and VIPs are treated as unknown first-time visitors, breaking automated arrival alerts and CRM history tracking². The architecture resolves this by decoupling physical hardware addresses from canonical customer identity records³. The platform treats MAC addresses as ephemeral transport pointers stored in a dedicated guest_devices table.

Identity verification is performed using verified phone numbers (via SMS one-time password) or authenticated OAuth tokens on the captive portal landing page³.

Once verified, the newly generated MAC address is linked to the primary guest_id record³. This composite mapping ensures that rotating MAC addresses continue to resolve to the diner's unified profile, preserving historical visit counts, loyalty tiers, preferences, and auto-tags³.

Asynchronous POS Disconnects, Post-Settlement Voids, and Loyalty Deficits

Consider a scenario where a diner settles a \$200 bill, earning 1,000 loyalty points (based on a 5x tier multiplier)¹². Thirty minutes later, floor staff realize a \$100 bottle of wine was billed incorrectly, void the line item on the POS terminal, and issue a partial refund⁷. If the loyalty engine does not track post-settlement adjustments, the diner retains unearned rewards, creating financial discrepancies⁷.

The POS webhook listener handles this by capturing the order.voided or order.refunded event, extracting the voided amount, reference check ID, and void reason code⁷. The loyalty engine calculates the required point clawback⁷:

$$\text{Clawback Points} = \text{Voided Amount} \times \text{Tier Multiplier} = 100 \times 5 = 500 \text{ points}$$

The system checks the guest's available balance:

- If $\text{points_balance} \geq 500$, the engine debits 500 points with reference REVERSAL_POST_VOID⁷.
- If the diner has already redeemed those points, the account balance shifts into a negative state ($\text{points_balance} = -500$) and the profile is flagged with UNRESOLVED_LOYALTY_DEFICIT⁷.

Subsequent point accruals are automatically applied toward resolving the negative balance

before positive liquid points accumulate, and the host console flags the account during subsequent visits³.

Split-Tender Settlements and Stored-Value Gift Card Timeouts

During folio settlement, a diner might split a \$150 bill across multiple payment methods: a \$50 digital gift card, 5,000 loyalty points (\$50 value), and the remaining \$50 charged to a physical credit card⁷. If the gift card balance endpoint times out mid-transaction, inconsistent payment states or double-charging could occur⁷.

The platform avoids partial payment failures by using a two-phase commit protocol for composite check payments:

1. **Phase 1 (Authorization Holds):** The system places 120-second cryptographic holds on the gift card balance and loyalty points balance without decrementing canonical funds⁷.
2. **Phase 2 (Capture & Settlement):** The POS processes the remaining credit card charge. Upon credit gateway approval, the platform executes atomic settlement calls across the gift card and loyalty ledgers simultaneously⁷.
3. **Rollback Handling:** If any tender verification fails (such as a gift card authorization timeout), all holds are released, the order ticket is marked PARTIALLY_PAID, and the POS displays an explicit error: Tender Failed: Gift Card Unreachable. Folio Remaining: \$100.00⁷.

Dwell Time Overruns and Escalation Pacing Alerts

When a dining party lingers at their table for 55 minutes—significantly exceeding the thirty-minute turnaround duration ($\Delta t_{TAT} = 30 \text{ minutes}$)—an incoming reservation scheduled for the same table could be left without seating¹.

The floor state engine manages pacing delays through multi-stage escalation thresholds¹:

- At $\Delta t_{elapsed} = 30 \text{ minutes}$, the host console table tile shifts to flashing amber, signaling staff to initiate the check presentation process¹.
- At $\Delta t_{elapsed} = 45 \text{ minutes}$, the table tile shifts to flashing red, triggering an audible alert on the host console and notifying the lead floor captain¹.
- The allocation engine evaluates floor capacity in real time¹. If another matching table is open, the system auto-migrates the incoming reservation to the vacant unit, updating the host run sheet without disrupting the arriving party¹.
- If the venue is operating at 100% capacity, the triage queue triggers an automated SMS to the arriving guest: Your table is undergoing final preparation. Enjoy a complimentary beverage at the bar while we finalize your seating.

Host Triage Queue Unresponsiveness and Grace Period Timeouts

During peak kitchen rushes, incoming digital reservation requests may sit unacknowledged in the host triage queue¹. Simultaneously, confirmed parties may fail to arrive, leaving vacant tables locked by no-show bookings¹.

The platform addresses these operational bottlenecks with automated timers:

- **Triage Auto-Escalation:** Reservations in the REQUESTED state carry a 300-second timer¹. If the host takes no action within five minutes, the system evaluates venue capacity: if candidate inventory is completely clear and the diner has a Good Standing profile tag, the system auto-confirms the booking and sends confirmation notifications¹. If inventory is tight, the request is escalated to a managerial console with high-priority visual alerts.
- **Grace Period Cancellation:** Confirmed bookings maintain a fifteen-minute arrival grace window ($t_{grace} = 15 \text{ minutes}$)¹. At $T_{start} + t_{grace}$, if the host has not marked the party as SEATED, a background worker marks the booking NO_SHOW, reverts the table status to AVAILABLE, and increments the guest's no-show counter in the CRM¹.

Network Partitions, Webhook Failures, and Reconciliation Polling

On-premise network interruptions can prevent POS terminals or captive Wi-Fi controllers from delivering transaction webhooks to the central platform, potentially causing missed spend data, out-of-sync loyalty balances, and inaccurate floor statuses¹.

The system maintains data consistency across network partitions through three architectural safeguards:

- **Idempotency & Replay Headers:** All webhook endpoints enforce idempotency keys (Idempotency-Key: {source}_{event_id}_{timestamp})²⁵. Duplicate webhooks delivered after network recovery are acknowledged without double-processing⁷.
- **Dead-Letter Queues (DLQ) & Exponential Backoff:** Failed incoming webhooks are routed to a persistent queue with exponential retry intervals (1s, 5s, 30s, 5m, 1h).
- **Nightly POS Audit Polling:** A nightly reconciliation worker queries the POS reporting API (GET /sales-report/v2/closed-orders?date={date})⁷. The script compares aggregated POS transactions against the platform's pos_orders database table⁷. Any missing checks are backfilled, loyalty points are recalculated and settled, and audit logs are generated for managerial review⁷.

Edge Case Scenario	Primary Risk / Failure Mode	Automated System Guardrail	Final System Outcome
Simultaneous Booking Collisions [cite: 1]	Double-allocation of table inventory to different parties ¹ .	Relational row-level locking (SELECT FOR UPDATE) within transaction ¹ .	First thread commits; second rolls back with 409 Conflict and alternatives ¹ .
Walk-In / Booking Conflict [cite: 1]	Walk-in seating overlaps with upcoming confirmed	Terminal checks: $t_{walkin} + \Delta t_{TAT} >$ ¹ .	Seating blocked; host directed to vacant alternate or waitlist ²³ .

	booking ¹ .		
Rotating Device MACs [cite: 3]	Returning diners treated as unknown first-time visitors ² .	Decouples MAC from primary identity; verifies phone via OTP ³ .	Ephemeral MAC bound to canonical guest_id; profile preserved ³ .
Post-Settlement POS Voids [cite: 7, 8]	Unearned loyalty points retained by customer after refund ⁷ .	POS void webhook listener calculates clawback: Void × ⁷ .	Debits points; flags LOYALTY_DEFICIT if balance is negative ⁷ .
Split-Payment Timeouts [cite: 7, 22]	Partial payment processing leaves folio unbalanced ⁷ .	Two-phase commit protocol with 120-second authorization holds ⁷ .	Failed tender releases holds; order marked PARTIALLY_PAID ⁷ .
Dwell Time Overruns [cite: 1]	Lingering party blocks table for incoming reservation ¹ .	Multi-stage visual alerts (amber at 30m, red at 45m) ¹ .	Auto-migrates incoming booking to open table or alerts host ¹ .
Host Queue Inaction [cite: 1]	Reservation requests sit unacknowledged during service rushes ¹ .	300-second triage timer triggers auto-confirmation or manager alert ¹ .	Prevents abandoned requests; ensures transparent booking status ¹ .
POS Network Outages [cite: 6]	Webhooks dropped during on-premise internet disconnects ⁶ .	Idempotent event consumer, Dead-Letter Queues, and nightly audit polling ⁷ .	Reconciles missed checks; backfills loyalty and spend data ⁷ .

Phased Implementation Roadmap and Architectural Conclusions

Building a resilient dual-sided restaurant management platform requires proving core transactional stability before introducing operational complexity¹. Hardcoding the initial table turnaround duration to thirty minutes establishes a predictable baseline for testing floor

pricing, host approval workflows, and database locking without the variance of dynamic dining durations¹.

Similarly, replacing automated device geolocation with fixed administrative localities keeps early engineering focused on data integrity and booking workflows rather than third-party mapping dependencies¹.

The platform architecture should advance across three structured implementation phases:

- **Phase 1: Transactional Core Validation:** Establish the normalized relational schema, implement database row-level locking to prevent conflicting bookings, and build the administrative dashboard with table status tiles, host acceptance queues, and live elapsed timers¹.
- **Phase 2: Operational Synchronization & Intelligence:** Transition the floor console from client polling to bidirectional WebSocket channels for real-time updates, integrate transactional notification gateways, deploy the captive Wi-Fi portal for automated CRM profile generation, and expand the turnaround model to accommodate party-size-dependent durations¹.
- **Phase 3: Enterprise Scale & Commerce Unification:** Replace database row locks with distributed Redis key leases for high-concurrency surge handling, integrate direct POS webhooks for real-time ticket and void tracking, and launch the multi-tiered membership engine with preloaded digital wallets and liquid loyalty points¹.

By addressing the core operational requirements—clear table capacities, reliable transactional locking, automated guest profiling, and direct host approval workflows—this specification equips engineering teams to build a platform that matches and exceeds the capabilities of industry benchmarks like Resy, SevenRooms, and Blackbird, providing an integrated operational foundation for modern restaurant hospitality¹.

Works cited

1. Restaurant POC PRD Creation.pdf
2. What Is a Restaurant CRM? Features, Data and How to Choose, <https://sevenrooms.com/blog/what-is-a-restaurant-crm/>
3. Guest WiFi for Restaurants | CaptiveWiFi, <https://www.captivewifi.com/for/restaurants>
4. FAQs - Blackbird, <https://www.blackbird.xyz/faqs>
5. How to setup the SevenRooms Connector with Captive WiFi, <https://help.captivewifi.io/integrations/how-to-setup-the-sevenrooms-connector-with-captive-wifi>
6. Restaurant API and Integrations - SevenRooms, <https://sevenrooms.com/platform/integrations-apis/>
7. Toast Refund Flows - Incentivio, <https://incentivio.zendesk.com/hc/en-us/articles/29765631467789-Toast-Refund-Flows>
8. View and manage Sales Report - Eats365 Knowledge Base, https://support.eats365pos.com/en_US/sales-report/view-and-manage-sales-report

[ort](#)

9. Restaurant CRM & Segmentation - SevenRooms, <https://sevenrooms.com/platform/crm/>
10. SevenRooms x The Grounds Hospitality Group · A Proposal, <https://www.holliesheehan.com/>
11. Void an order - - Developer guide - Toast platform docs -, <https://doc.toasttab.com/doc/devguide/apiVoidOrder.html>
12. Everything you need to know about Blackbird's dining loyalty program, <https://thepointsguy.com/loyalty-programs/blackbird-dining-rewards-program/>
13. 10 Best Restaurant CRM Systems Compared - Eat App, <https://restaurant.eatapp.co/blog/10-best-restaurant-crm-systems-compared>
14. MINA Group: Book of Law - PDFCOFFEE.COM, <https://pdfcoffee.com/mina-group-book-of-law-pdf-free.html>
15. Blackbird, <https://www.blackbird.xyz/>
16. 10 Tools Compared: Best Gastro Software (2026) - Gitnux, <https://gitnux.org/best/gastro-software/>
17. Aldelo Pos User Manual (English) | PDF | Point Of Sale - Scribd, <https://www.scribd.com/document/423939066/Aldelo-Pos-User-Manual-English>
18. Restaurant Software Development | POS & Ordering - Algovaani, <https://algovaani.com/restaurant-software>
19. Restaurant loyalty program Blackbird launches payments, <https://www.fastcompany.com/91162243/restaurant-loyalty-program-blackbird-is-launching-payments-and-introducing-guest-value-scores>
20. Integrations | Captive WiFi Knowledgebase, <https://help.captivewifi.io/integrations>
21. Blackbird - My Framer Site - Studio Salt, <https://studiosalt.co/projects/blackbird>
22. 4. Inside Blackbird: A Web3 Loyalty App Built for Mainstream, <https://onchain.org/research/crypto-consumer-apps-unlocking-real-world-utility-and-driving-mass-adoption-in-2025/chapter/4/>
23. 18 Restaurant Technology Tools You Need to Power Your Business, <https://sevenrooms.com/blog/15-restaurant-industry-technology-tools-you-need-to-power-your-business/>
24. XenosHMS, <https://cloud.xenoshms.com/>
25. Sevenrooms | APIs.io Providers, <https://apis.io/providers/sevenrooms/>